Joobsheet 8

2.1 Percobaan 1: Review Perulangan 1

Waktu Percobaan: 30 menit

Percobaan ini mengulang konsep perulangan FOR sebelum mempelajari perulangan bersarang. Program menampilkan karakter `*` sebanyak *n* kali secara mendatar (ke arah samping). Gunakan input *n* berupa bilangan bulat positif.

Langkah-langkah Percobaan

1. Buat repository baru bernama **daspro-jobsheet8** di GitHub, *clone* ke komputer Anda, lalu buka foldernya di Visual Studio Code.
2. Buat file baru **StarNoAbsen.java** di dalam folder tersebut. Ganti NoAbsen dengan nomor absen Anda, misalnya **Star05.java**.
3. Tambahkan sintaks import class `Scanner` pada baris paling atas program.

```
import java.util.Scanner;
```

4. Buat class dan fungsi/method `main()`.

```
public class StarNoAbsen {
    public static void main(String[] args) {

    }

}
```

5. Di dalam method `main()`, deklarasikan objek `Scanner` dengan nama `sc`, kemudian tampilkan instruksi untuk memasukkan nilai yang akan disimpan ke variabel `n`.

```
Scanner sc = new Scanner(System.in);
System.out.print("Masukkan nilai n = "); int
n = sc.nextInt();
```

6. Pada baris selanjutnya, buat perulangan FOR untuk mencetak `*` sebanyak *n* kali. Perhatikan bahwa yang digunakan adalah `print`, **bukan** `println`, karena karakter `*` ingin ditampilkan berderet tanpa pindah baris.

```
for (int i = 1; i <= n; i++) {

    System.out.print("*");
}
```

7. Setelah perulangan selesai, tambahkan `System.out.println();` agar kursor pindah ke baris baru, lalu tutup objek Scanner dengan `sc.close();`. Kode lengkap program menjadi seperti berikut.

```java
import java.util.Scanner;

public class StarNoAbsen {
    public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
System.out.print("Masukkan nilai n = ");
int n = sc.nextInt();

        for (int i = 1; i <= n; i++) {
            System.out.print("*");
        }
        System.out.println();
sc.close();
    }
}
```

8. Simpan file, kemudian *compile* dan jalankan program.
9. Masukkan n = 5. Amati hasilnya; output harus serupa dengan tampilan berikut (teks tebal biru adalah input yang diketik melalui *keyboard*).

```
Masukkan nilai n = 5
*****
```

10. Jalankan kembali program dengan n = 3 dan n = 7. Catat jumlah karakter `*` yang muncul pada setiap pengujian.
11. Setelah program berjalan sesuai harapan, *commit* dan *push* file **StarNoAbsen.java** ke repository **daspro-jobsheet8**.

Pertanyaan

1. Jika inisialisasi `i = 1` diubah menjadi `i = 0` tanpa mengubah kondisi, berapa karakter `*` yang tercetak ketika n = 5? Jelaskan.
2. Jika kondisi `i <= n` diganti menjadi `i < n`, berapa kali perulangan dijalankan saat n = 5? Mengapa hasilnya berbeda?
3. Prediksi output potongan kode berikut ketika n = 9, kemudian buktikan dengan mengganti perulangan FOR pada program Anda dan menjalankannya.

```java
for (int i = 2; i <= n; i += 2) {

    System.out.print(i + " "); }
```

Modifikasi kode tersebut agar mencetak bilangan **ganjil** positif yang tidak melebihi n.

4. Jelaskan mengapa perubahan *increment* `i++` menjadi `i--` (dengan kondisi tetap `i <= n`) membuat perulangan berjalan terus-menerus dan tidak berhenti dalam waktu wajar untuk n positif. Jangan menjalankan versi tersebut; jika terlanjur berjalan, hentikan program dengan menekan **Ctrl + C** pada terminal.

Jawaban:

1. Jika inisialisasi int i = 1 diubah menjadi int i = 0 (kondisi tetap i <= n dan $n = 5$):

- **Jumlah karakter * yang tercetak: 6 karakter * (******).**
- **Penjelasan: Perulangan dimulai dari indeks $i = 0$hingga $i = 5$. Langkah perulangannya adalah: $i = 0, 1, 2, 3, 4, 5$. Total perulangan berjalan sebanyak 6 kali ($5 - 0 + 1 = 6$), sehingga bintang yang dicetak bertambah satu dari jumlah nilai n.**

2. Jika kondisi i <= n diganti menjadi i < n (dengan $n = 5$):

- **Jumlah perulangan dijalankan: 4 kali ($i = 1, 2, 3, 4$).**
- **Alasan perbedaan:**
  - **Pada kondisi i <= n, ketika $i = 5$, pernyataan $5 \leq 5$bernilai true (benar), sehingga bintang ke-5 tetap dicetak.**
  - **Pada kondisi i < n, ketika $i = 5$, pernyataan $5 < 5$bernilai false (salah). Akibatnya, perulangan langsung berhenti sebelum mencetak bintang ke-5.**

3. Prediksi output potongan kode saat $n = 9$dan modifikasinya:

- **Prediksi Output ($n = 9$):**

Plaintext

2 4 6 8

  - **Penjelasan: Perulangan dimulai dari $i = 2$dan bertambah 2 setiap iterasi (i += 2). Angka genap ≤ 9yang memenuhi adalah 2, 4, 6, dan 8.**

- **Modifikasi Kode untuk Mencetak Bilangan Ganjil Positif ($\leq n$): Ganti inisialisasi awal menjadi $i = 1$dengan inkremen tetap i += 2:**

Java

```
for (int i = 1; i <= n; i += 2) {
   System.out.print(i + " ");
}
```

(Jika $n = 9$, kode modifikasi ini akan mencetak output: 1 3 5 7 9)

4. Penjelasan Mengapa Perubahan i++ Menjadi i-- Menyebabkan *Infinite Loop*:

- **Penjelasan:**
  - **Apabila nilai awal $i = 1$dan $n = 5$(bilangan positif), kondisi perulangan yang dipersyaratkan adalah i <= n ($i \leq 5$).**
  - **Jika menggunakan *decrement* i--, nilai *i*akan terus berkurang pada setiap iterasi: $i = 1, 0, -1, -2, -3$, ...dan seterusnya.**

- **Karena nilai *i*semakin mengecil, maka nilainya akan selalu lebih kecil dari *n*(5), sehingga syarat i <= 5 akan selalu bernilai true (benar) dan perulangan tidak akan pernah bernilai false untuk menghentikan program (*infinite loop*).**
-



```
Praktek Daspro > Jobsheet 8 > Star01.java > ...
1   import java.util.Scanner;
2
3   public class Star01 {
        Run | Debug
4       public static void main(String[] args) {
5           Scanner sc = new Scanner(System.in);
6           System.out.print("Masukkan nilai n = ");    "Masukkan": Unknown word.
7           int n = sc.nextInt();
8
9           for (int i = 1; i <= n; i++) {
10              System.out.print("*");
11          }
12
13          System.out.println();
14
15          //Modifikasi    "Modifikasi": Unknown word.
16
17          System.out.print("Masukkan nilai m = ");    "Masukkan": Unknown word.
18          int m = sc.nextInt();
19
20          for (int i = 1; i <= m; i += 2) {
21              System.out.print(i + " ");
22          }
23
24          System.out.println();
25          sc.close();
26      }
27  }
```



```
Problems 234   Output   Debug Console   Terminal   Ports   DevDb   Spell Checker 211   Azure

PS C:\Matkul\Daspro & Prakdaspro> ^C
PS C:\Matkul\Daspro & Prakdaspro>
PS C:\Matkul\Daspro & Prakdaspro>  c:; cd 'c:\Matkul\Daspro & Prakdaspro'; & 'C:\Users\DELL\.vscode
e' '' XX:+ShowCodeDetailsInExceptionMessages' '' cp' 'C:\Users\DELL\AppData\Roaming\Code\User\workspa
pro_22a6ac91\bin' 'Star01'
Masukkan nilai n = 4
****
Masukkan nilai m = 9
1 3 5 7 9
PS C:\Matkul\Daspro & Prakdaspro>
```

2.2 Percobaan 2: Pola Bintang Persegi Panjang (Nested FOR)

Waktu Percobaan: 30 menit

Pada percobaan ini digunakan *nested loop* FOR untuk mencetak persegi panjang bintang. Jumlah baris dan kolom dimasukkan secara terpisah sehingga peran kedua loop dapat diamati dengan

jelas: *outer loop* mengatur **baris**, sedangkan *inner loop* mengatur **kolom** (ingat analogi rak/loker pada materi).

Contoh input baris = 3 dan kolom = 6 menghasilkan output:

```
******
******
******
```

Langkah-langkah Percobaan

1. Buat file baru **RectangleNoAbsen.java**.
2. Tambahkan `import java.util.Scanner;`, kemudian buat class `RectangleNoAbsen` beserta method `main()` seperti pada Percobaan 1.
3. Di dalam `main()`, deklarasikan Scanner `sc`, lalu baca dua bilangan bulat `baris` positif, yaitu dan `kolom`.

```
        Scanner sc = new Scanner(System.in);
System.out.print("Masukkan jumlah baris: ");
int baris = sc.nextInt();
System.out.print("Masukkan jumlah kolom: ");
int kolom = sc.nextInt();
```

4. Tuliskan terlebih dahulu perulangan yang mencetak `*` sebanyak `kolom` kali (sama seperti Percobaan 1). Perulangan ini nantinya menjadi *inner loop*.

```
for (int j = 1; j <= kolom; j++) {

    System.out.print("*"); }
```

5. Jalankan program dengan input baris = 3 dan kolom = 6. Perhatikan bahwa yang tercetak baru **satu baris** bintang, karena belum ada perulangan untuk baris.

```
Masukkan jumlah baris: 3
Masukkan jumlah kolom: 6
******
```

6. Bungkus perulangan tersebut dengan *outer loop* yang berulang sebanyak `baris` kali. Tambahkan `System.out.println();` **setelah** *inner loop* (masih di dalam *outer loop*) agar setiap baris bintang dicetak pada baris baru.

```
for (int i = 1; i <= baris; i++) {       // outer loop: baris     for (int
j = 1; j <= kolom; j++) {   // inner loop: kolom
        System.out.print("*");
    }

    System.out.println();                // pindah baris

}
```

7. Tutup Scanner dengan `sc.close();` di akhir method `main()`. Kode lengkap program menjadi seperti berikut.

```java
import java.util.Scanner;

public class RectangleNoAbsen {
    public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
        System.out.print("Masukkan jumlah baris: ");
int baris = sc.nextInt();
        System.out.print("Masukkan jumlah kolom: ");
int kolom = sc.nextInt();

        for (int i = 1; i <= baris; i++) {
for (int j = 1; j <= kolom; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
        sc.close();
    }
}
```

8. Simpan, *compile*, dan jalankan program dengan input baris = 3 dan kolom = 6. Output harus serupa dengan tampilan berikut.

```
Masukkan jumlah baris: 3
Masukkan jumlah kolom: 6
******
****** ******
```

9. Jalankan kembali untuk input (baris, kolom) = (4, 2) dan (1, 5). Bandingkan bentuk ketiga output tersebut.

Pertanyaan

1. Tentukan fungsi variabel `i` dan `j` pada program tersebut.
2. Berapa kali *outer loop*, *inner loop*, dan perintah `print("*")` dijalankan untuk baris = 4 dan kolom = 7?

3. Apa yang terjadi jika batas perulangan luar dan dalam ditukar (outer menggunakan `kolom`, inner menggunakan `baris`)? Gunakan baris = 3 dan kolom = 6 untuk menjelaskan. 4. Jelaskan akibat penghapusan `System.out.println();` setelah *inner loop*.

5. Modifikasi program agar karakter yang dicetak adalah `#`, lalu *commit* dan *push* perubahan ke GitHub.

Jawaban:

1. Fungsi variabel i dan j pada program tersebut:

- **Variabel i: Berfungsi sebagai pengontrol pada outer loop (perulangan luar) untuk menghitung dan melacak iterasi jumlah baris yang sedang dikerjakan.**

- **Variabel j: Berfungsi sebagai pengontrol pada inner loop (perulangan dalam) untuk menghitung dan melacak iterasi jumlah kolom (pencetakan karakter bintang) pada setiap baris tertentu.**

2. Jumlah eksekusi saat baris = 4 dan kolom = 7:

- **Outer loop dijalankan: 4 kali (karena jumlah baris ada 4).**
- **Inner loop dijalankan: 4 × 7 = 28 kali secara total (karena untuk setiap 1 baris, inner loop berjalan 7 kali penuh).**
- **Perintah System.out.print("*") dijalankan: 28 kali (sama dengan total eksekusi inner loop, menghasilkan total 28 buah bintang).**

3. Apa yang terjadi jika batas perulangan luar dan dalam ditukar (baris = 3 dan kolom = 6)?

- **Penjelasan: Jika outer loop menggunakan variabel kolom (batas 6) dan inner loop menggunakan variabel baris (batas 3), maka perulangan luar akan berjalan sebanyak 6 kali (mencetak 6 baris ke bawah), dan di setiap barisnya inner loop akan mencetak bintang sebanyak 3 kali ke samping.**
- **Bentuk Output:**

Plaintext

***

***

***

***

***

***

(Bentuk persegi panjangnya berubah ukuran menjadi 6 baris ke bawah dan 3 kolom ke samping).

4. Akibat penghapusan System.out.println(); setelah inner loop:

- **Penjelasan: Perintah System.out.println(); berfungsi untuk memindahkan kursor ke baris berikutnya setelah satu baris bintang selesai dicetak. Jika baris ini dihapus, seluruh karakter bintang dari baris pertama hingga baris terakhir akan tercetak menyatu berderet memanjang ke samping (horizontal) pada satu baris yang sama tanpa ada jeda ganti baris.**

5. Modifikasi Program (Karakter dicetak #):

Berikut adalah kode program setelah dimodifikasi agar mencetak karakter # alih-alih *:

```java
import java.util.Scanner;

public class Rectangle01 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Masukkan jumlah baris: ");
        int baris = sc.nextInt();

        System.out.print("Masukkan jumlah kolom: ");
        int kolom = sc.nextInt();

        for (int i = 1; i <= baris; i++) {
            for (int j = 1; j <= kolom; j++) {
                System.out.print("*");
            }

            System.out.println();
        }


        //Modifikasi
        System.out.print("Masukkan jumlah baris: ");
        int baris1 = sc.nextInt();

        System.out.print("Masukkan jumlah kolom: ");
        int kolom1 = sc.nextInt();

        for (int i = 1; i <= baris1; i++) {
            for (int j = 1; j <= kolom1; j++) {
                System.out.print("#"); // Karakter diubah menjadi pagar (#)
            }
            System.out.println();
        }
        sc.close();
    }
}
```

```
PS C:\Matkul\Daspro & Prakdaspro>  c:; cd 'c:\Matkul\Daspro & Prakdas
e' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\DELL\App
pro_22a6ac91\bin' 'Rectangle01'
Masukkan jumlah baris: 3
Masukkan jumlah kolom: 6
******
******
******
Masukkan jumlah baris: 7
Masukkan jumlah kolom: 4
####
####
####
####
####
####
####
PS C:\Matkul\Daspro & Prakdaspro>
```

2.3 Percobaan 3: Debugging Bintang Segitiga – Nested WHILE

Waktu Percobaan: 45 menit

Kode awal pada percobaan ini sengaja dibuat belum menghasilkan format segitiga yang benar. Tugas Anda adalah mengidentifikasi kesalahannya dan memperbaikinya tanpa mengganti struktur WHILE menjadi FOR.

Output target untuk n = 5:

```
*
**
***
****
*****
```

Langkah-langkah Percobaan

1. Buat file baru **TriangleNoAbsen.java**.
2. Ketik kode awal berikut secara lengkap (sesuaikan nama *class* dengan nama file Anda).

```
import java.util.Scanner;

public class TriangleNoAbsen {
    public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
System.out.print("Masukkan nilai n = ");
int n = sc.nextInt();

        int i = 0;
while (i <= n) {
int j = 0;
while (j < i) {
                System.out.print("*");
j++;             }             i++;
}
        sc.close();
    }
}
```

3. Simpan dan jalankan program dengan n = 3, kemudian dengan n = 5. Catat output aktual yang muncul, lalu bandingkan dengan output target.
4. Untuk mencari penyebab kesalahan, tambahkan perintah bantu (debug) sementara tepat setelah baris `while (i <= n) {`. Perintah ini menampilkan nilai `i` setiap kali *outer loop* dimulai.

```
while (i <= n) {

    System.out.print("[i=" + i + "] ");    // debug sementara
int j = 0;
```

5. Jalankan kembali program dengan n = 5, lalu amati hasilnya:

a. berapa kali *outer loop* dijalankan? Apakah sama dengan jumlah baris yang diinginkan?

b. berapa karakter `*` yang tercetak saat `i = 0`?

c. mengapa semua karakter tampil dalam satu baris?

6. Perbaiki kode satu per satu dan jalankan program setelah setiap perbaikan:

a. tambahkan `System.out.println();` setelah *inner loop* selesai, sebelum `i++`. Jalankan dan amati: apakah masih ada baris yang tidak berisi `*`?

b. ubah inisialisasi dan/atau kondisi *outer loop* agar perulangan berjalan tepat *n* kali tanpa baris kosong. Jalankan kembali.

7. Setelah output sama dengan output target, **hapus** baris *debug*. Jalankan sekali lagi untuk memastikan hasilnya, lalu *commit* dan *push* ke repository Anda.

8. Modifikasi program agar **setelah** segitiga di atas tercetak, program juga mencetak segitiga terbalik. Tuliskan nested WHILE kedua di bawah nested WHILE pertama (masih di dalam `main()`), tanpa mengganti WHILE menjadi FOR.

> **Catatan:** variabel `i` sudah dideklarasikan sebelumnya, jadi pada *nested* WHILE kedua cukup tulis `i = n`; tanpa `int`. Menulis `int i = n;` akan menyebabkan *error*: *variable i is already defined*.

9. Jalankan program dengan n = 5. Bagian segitiga terbalik harus serupa dengan tampilan berikut.

```
*****
****
***
**
*
```

Pertanyaan

1. Sebutkan ketidaksesuaian output awal dan minimal dua perbaikan yang diperlukan untuk menghasilkan pola segitiga bertingkat.
2. Jelaskan peran variabel `i` dan `j`. Mengapa nilai `j` harus diinisialisasi kembali ke 0 pada setiap iterasi *outer loop*? Apa yang terjadi jika `j` tidak di-*reset*?
3. Jelaskan perubahan inisialisasi, kondisi, dan perubahan nilai (*update*) pada *outer loop* yang diperlukan untuk membentuk segitiga terbalik.

Jawaban:

1. Ketidaksesuaian output awal dan dua perbaikan yang diperlukan:

- **Ketidaksesuaian Output Awal:**
  - **Muncul baris kosong ekstra di awal karena inisialisasi $i = 0$(saat $i = 0$, inner loop tidak berjalan namun i++ tetap memicu baris baru/tidak ada cetakan yang benar).**
  - **Semua karakter bintang tercetak menyatu dalam satu baris horizontal karena perintah System.out.println() belum ada di dalam *outer loop*.**

- **Dua Perbaikan Utama:**
    1. **Menambahkan perintah System.out.println(); setelah *inner loop* selesai agar setiap baris berpindah ke baris baru.**
    2. **Mengubah inisialisasi awal *outer loop* dari int i = 0 menjadi int i = 1 (atau mengubah kondisi loop) agar perulangan berjalan tepat sebanyak $n$baris tanpa menghasilkan baris kosong di awal.**

2. Peran variabel i dan j, serta alasan reset nilai j:

- **Peran Variabel i: Mengontrol *outer loop* untuk menentukan jumlah baris yang sedang diproses.**
- **Peran Variabel j: Mengontrol *inner loop* untuk mencetak jumlah bintang (*) pada baris tertentu sesuai dengan nilai i.**
- **Mengapa j harus di-reset ke 0 pada setiap iterasi *outer loop*: Variabel j bertindak sebagai penghitung (*counter*) kolom yang bertambah dari 0 hingga mencapai batas i. Jika j tidak di-reset kembali ke 0 di awal setiap baris baru, maka pada baris berikutnya nilai j akan tetap besar (tidak mulai dari 0 lagi), sehingga *inner loop* tidak akan berjalan dan bintang gagal dicetak pada baris-baris selanjutnya.**

3. Perubahan inisialisasi, kondisi, dan update untuk membentuk segitiga terbalik:

Untuk membentuk segitiga terbalik (dari jumlah bintang terbanyak $n$turun hingga 1):

- **Inisialisasi: Nilai awal *outer loop* diatur dari nilai maksimum, yaitu $i = n$(atau $i = 5$jika $n = 5$).**
- **Kondisi: Batas perulangan diperiksa selama $i \geq 1$(selama $i$lebih besar dari atau sama dengan 1).**
- **Update (Perubahan Nilai): Menggunakan operasi decrement (i--) pada setiap akhir iterasi *outer loop* agar nilai $i$terus berkurang dari $n$turun hingga mencapai 1.**

```java
import java.util.Scanner;

public class Triangle01 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Masukkan nilai n = ");
        int n = sc.nextInt();

        int i = 1;
        while (i <= n) {
            int j = 0;
            while (j < i) {
                System.out.print("*");
                j++;
            }
            System.out.println();
            i++;
        }

        System.out.println();

        i = n;
        while (i >= 1) {
            int j = 0;
            while (j < i) {
                System.out.print("*");
                j++;
            }
            System.out.println();
            i--;
        }

        sc.close();
    }
}
```

```
Problems 263  Output  Debug Console  Terminal  Ports  DevDb  Sp

Masukkan nilai n = 4
*
**
***
****

****
***
**
*
PS C:\Matkul\Daspro & Praktdaspro>
```

2.4 Percobaan 4: Studi Kasus Nilai Tugas Proyek Kelompok

Waktu Percobaan: 75 menit

Dalam penilaian proyek kelompok, setiap kelompok memperoleh nilai dari sejumlah penilai. Program menerima jumlah kelompok dan jumlah penilai yang sama untuk setiap kelompok, kemudian menghitung rata-rata nilai tiap kelompok dan menentukan kelompok dengan ratarata tertinggi. Kasus ini diselesaikan dengan *nested loop* tanpa perlu menyimpan seluruh nilai: **WHILE**

(luar) untuk kelompok, **FOR** (tengah) untuk penilai, dan **DO-WHILE** (dalam) untuk mengulang input nilai yang tidak valid.

Langkah-langkah Percobaan

1. Buat file baru **NilaiKelompokNoAbsen.java** yang berisi `import java.util.Scanner;`, class, method `main()`, dan deklarasi Scanner dengan nama `sc`.
2. Minta input jumlah kelompok dan jumlah penilai. Keduanya harus lebih besar dari 0; gunakan IF-ELSE sehingga seluruh proses penilaian hanya dijalankan pada blok `else`.

```
System.out.print("Masukkan jumlah kelompok : "); int
jumlahKelompok = sc.nextInt();
System.out.print("Masukkan jumlah penilai  : "); int
jumlahPenilai = sc.nextInt();

if (jumlahKelompok <= 0 || jumlahPenilai <= 0) {
    System.out.println("Jumlah kelompok dan penilai harus > 0.");
} else {
    // langkah 3 sampai 8 ditulis di dalam blok else ini
}
sc.close();
```

3. Di dalam blok `else`, siapkan variabel yang nilainya harus **bertahan** selama semua kelompok diproses.

```
int nomorKelompok = 1;         // penghitung outer loop double
rataTertinggi = -1;     // -1 agar kelompok pertama pasti tercatat int
kelompokTerbaik = 0;
```

4. Buat *outer loop* menggunakan WHILE untuk memproses kelompok satu per satu. Pada awal setiap kelompok, tampilkan nomor kelompok dan *reset* `totalNilai` menjadi 0. Jangan lupa `nomorKelompok++` di akhir badan perulangan; tanpa baris ini perulangan tidak akan berhenti.

```
while (nomorKelompok <= jumlahKelompok) {
    System.out.println("Kelompok " + nomorKelompok);
double totalNilai = 0;

    // inner loop penilai (langkah 5)

    // hitung rata-rata dan cek kelompok terbaik (langkah 6-7)

    nomorKelompok++;
}
```

5. Ganti komentar *inner loop penilai* dengan perulangan FOR untuk memasukkan nilai dari setiap penilai dan menjumlahkannya ke `totalNilai`. Nilai valid berada pada rentang 0–100; gunakan DO-WHILE agar input nilai yang sama diminta kembali selama nilainya di luar rentang.

```
for (int j = 1; j <= jumlahPenilai; j++) {
int nilai;     do {
        System.out.print("  Nilai dari Penilai " + j + ": ");
nilai = sc.nextInt();
        if (nilai < 0 || nilai > 100) {
            System.out.println("  Nilai harus 0-100. Ulangi input!");
}
    } while (nilai < 0 || nilai > 100);
totalNilai += nilai;
}
```

6. Setelah *inner loop* selesai, hitung rata-rata menggunakan tipe `double` dan tampilkan hasilnya.

```
double rataNilai = totalNilai / jumlahPenilai;
System.out.println("Kelompok " + nomorKelompok
        + ": nilai rata-rata = " + rataNilai);
```

7. Bandingkan `rataNilai` dengan `rataTertinggi`. Lengkapi sendiri isi blok IF berikut. Jika rata-rata sama, kelompok yang ditemukan lebih awal tetap menjadi pilihan (karena digunakan operator `>`, bukan `>=`).

```
if (rataNilai > rataTertinggi) {
    // lengkapi: perbarui rataTertinggi dan kelompokTerbaik
}
```

8. Setelah kurung kurawal penutup WHILE (masih di dalam blok `else`), tampilkan kelompok terbaik dan rata-rata tertinggi.

```
System.out.println("==============================");
System.out.println("Kelompok terbaik    : " + kelompokTerbaik);
System.out.println("Rata-rata tertinggi : " + rataTertinggi);
```

9. Simpan, *compile*, dan jalankan program dengan 3 kelompok dan 2 penilai. Uji juga validasi dengan sengaja memasukkan nilai 105. Output harus serupa dengan tampilan berikut.

```
Masukkan jumlah kelompok : 3
Masukkan jumlah penilai  : 2
Kelompok 1
  Nilai dari Penilai 1: 80
  Nilai dari Penilai 2: 90
Kelompok 1: nilai rata-rata = 85.0
Kelompok 2
  Nilai dari Penilai 1: 75
  Nilai dari Penilai 2: 105
  Nilai harus 0-100. Ulangi input!
Nilai dari Penilai 2: 85
Kelompok 2: nilai rata-rata = 80.0
Kelompok 3
  Nilai dari Penilai 1: 90
  Nilai dari Penilai 2: 96
```

```
Kelompok 3: nilai rata-rata = 93.0
==============================
Kelompok terbaik    : 3
Rata-rata tertinggi : 93.0
```

10. Jalankan kembali dengan variasi lain, misalnya 2 kelompok dan 3 penilai, serta uji input tidak valid (jumlah kelompok = 0) untuk memastikan pesan kesalahan tampil.
11. *Commit* dan *push* kode program ke GitHub.

Pertanyaan

1. Mengapa `totalNilai` perlu di-*reset* di dalam *outer loop*, tetapi `rataTertinggi` tidak boleh di-*reset* pada setiap kelompok?
2. Untuk 3 kelompok dan 2 penilai (tanpa input yang salah), berapa kali badan perulangan FOR dijalankan secara keseluruhan?
3. Apa yang terjadi jika `totalNilai` dideklarasikan sebagai `int` dan rata-rata dihitung menggunakan pembagian dua bilangan bulat? Berikan contoh nilai.
4. Mengapa `rataTertinggi` diberi nilai awal -1, bukan 0? Berikan contoh input yang membuat hasil salah apabila nilai awalnya 0.
5. Program ini memiliki tiga tingkat perulangan. Sebutkan masing-masing jenis perulangan beserta perannya.

Jawaban:

1. Mengapa totalNilai perlu di-reset di dalam outer loop, tetapi rataTertinggi tidak?

- **totalNilai harus di-reset menjadi 0 pada setiap awal iterasi kelompok baru agar akumulasi nilai penilai antar kelompok tidak saling bercampur (murni menghitung nilai kelompok yang sedang berjalan saja).**
- **rataTertinggi tidak boleh di-reset di dalam kelompok karena variabel ini bertindak sebagai penyimpan rekor global untuk membandingkan nilai terbaik dari kelompok pertama hingga kelompok terakhir. Jika di-reset setiap kelompok, perbandingan rekor tertinggi akan hilang.**

2. Berapa kali badan perulangan FOR dijalankan untuk 3 kelompok dan 2 penilai?

- **6 kali secara keseluruhan.**
- ***Penjelasan:* Karena ada 3 kelompok (outer loop berjalan 3 kali) dan setiap kelompok memiliki 2 penilai (inner loop FOR berjalan 2 kali per kelompok), maka total eksekusi perulangan FOR adalah $3 \times 2 = 6$ kali.**

3. Apa yang terjadi jika totalNilai dideklarasikan sebagai int dan rata-rata dihitung menggunakan pembagian bilangan bulat?

- **Terjadi pembagian bulat (*integer division*), sehingga angka desimal di belakang koma akan hilang (*truncated*).**

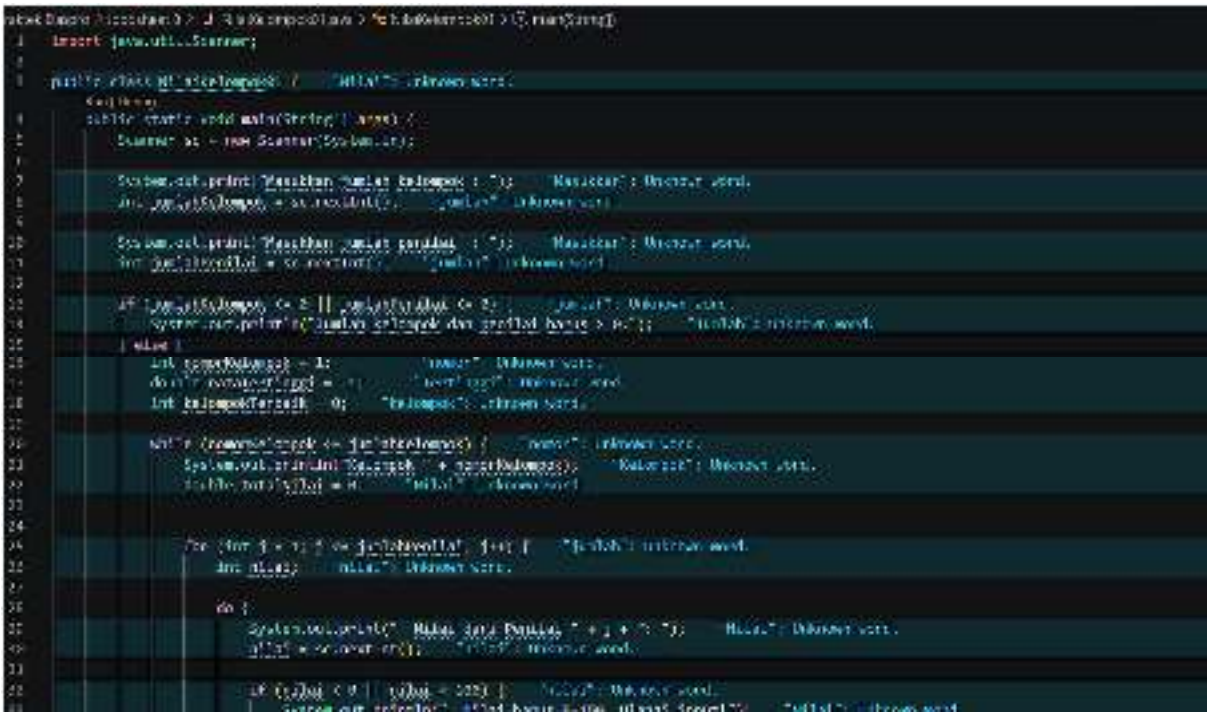
- *Contoh:* **Jika total nilai adalah 85 dari 2 penilai (seharusnya $85/2 = 42.5$), karena menggunakan tipe int (misal 85 / 2), hasilnya akan dibulatkan menjadi 42 saja, bukan 42.5.**

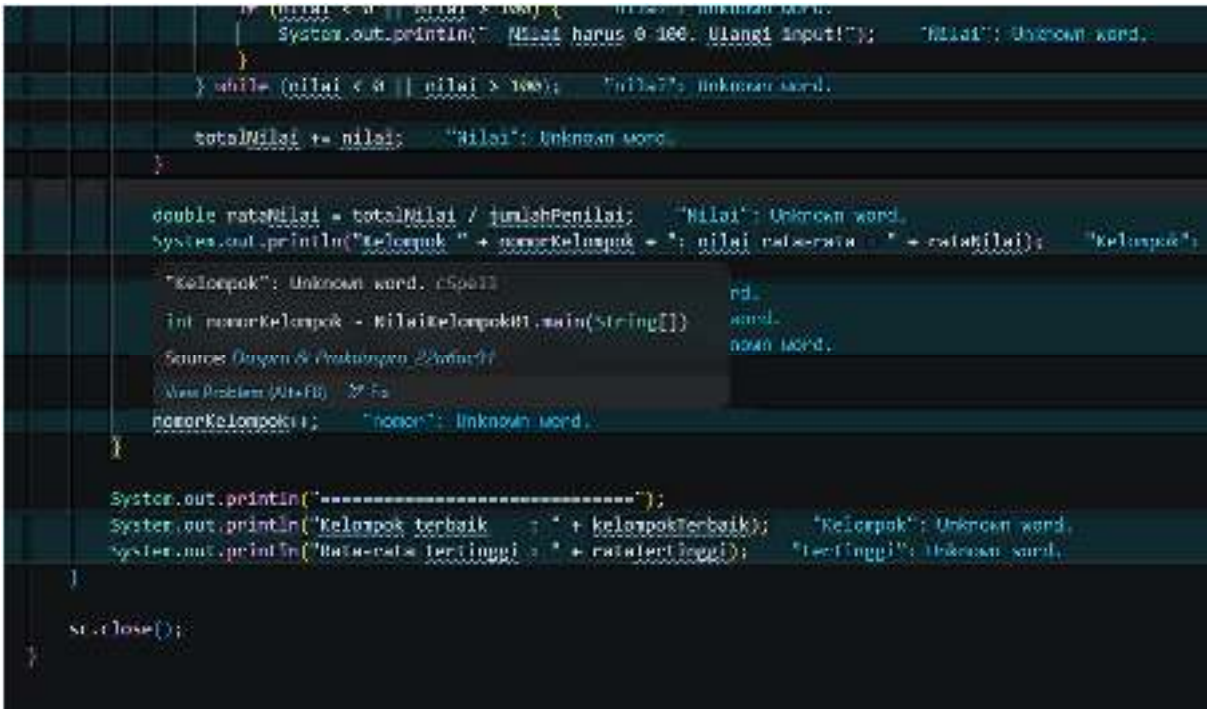
4. Mengapa rataTertinggi diberi nilai awal -1, bukan 0? Berikan contoh input yang membuat hasil salah jika nilai awalnya 0.

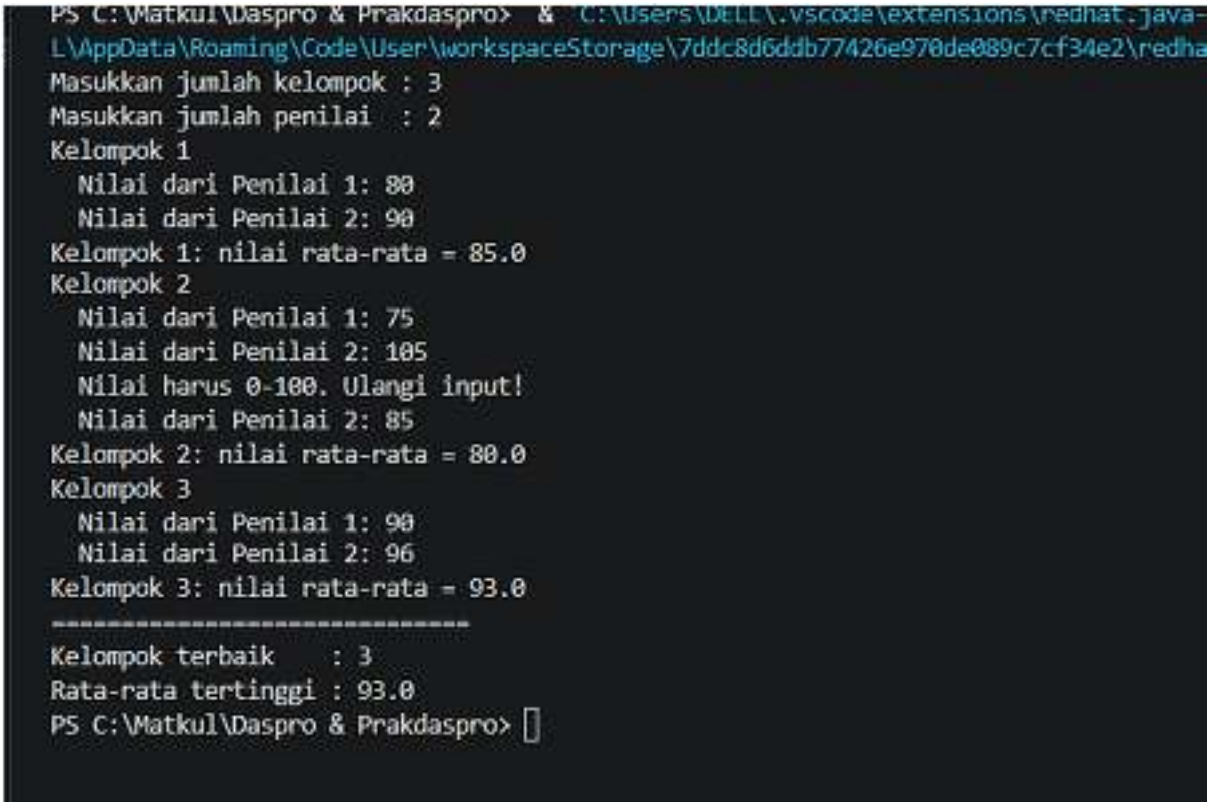
- **Diberi nilai awal -1 agar nilai rata-rata dari kelompok pertama (meskipun kecil, misalnya di atas 0) pasti bisa mengalahkan nilai awal -1 tersebut sehingga langsung tercatat sebagai rekor pertama.**
- **Contoh Kasus Salah jika Nilai Awal 0: Jika seluruh kelompok yang dinilai ternyata mendapat rata-rata di bawah 0 (misalnya kelompok 1 mendapat rata-rata -5, atau rentang nilai diizinkan negatif), jika inisialisasi awal 0, maka variabel rataTertinggi tidak akan tergantikan oleh kelompok manapun karena nilai awalnya sudah lebih besar dari rata-rata kelompok.**

5. Sebutkan tiga tingkat perulangan beserta perannya dalam program ini:

1. **WHILE (Outer Loop): Berperan untuk mengatur jalannya pemrosesan per kelompok dari kelompok 1 hingga kelompok terakhir (jumlahKelompok).**
2. **FOR (Inner Loop Menengah): Berperan untuk mengatur iterasi pengambilan nilai dari masing-masing penilai (jumlahPenilai) di setiap kelompok.**
3. **DO-WHILE (Inner Loop Terdalam): Berperan untuk melakukan validasi input nilai agar berulang meminta masukan dari pengguna apabila nilai yang dimasukkan berada di luar rentang valid ($0 - 100$).**







1. Tugas 1: Pola Bilangan Bertingkat

Buat program untuk mencetak pola bilangan 1 sampai nomor baris. Masukkan bilangan bulat n (n minimal 1). Pada baris pertama tercetak 1, pada baris kedua 1 2, dan seterusnya sampai baris ke-n. Setelah pola selesai, tampilkan total banyaknya angka yang telah dicetak. Simpan dengan nama file: **Tugas1NoAbsen.java**. Contoh untuk n = 5:

```
Masukkan nilai n = 5
```

```
1
```

1 2

```
1 2 3
1 2 3 4
1 2 3 4 5
Total angka tercetak: 15
```

Jawaban:



```
import java.util.Scanner;

public class Tugas1_01 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Masukkan nilai n = ");
        int n = sc.nextInt();

        if (n < 1) {
            System.out.println("Nilai n harus minimal 1.");
        } else {
            int totalAngka = 0;

            for (int i = 1; i <= n; i++) {
                for (int j = 1; j <= i; j++) {
                    System.out.print(j + " ");
                    totalAngka++;
                }

                System.out.println();
            }

            System.out.println("Total angka tercetak: " + totalAngka);
        }

        sc.close();
    }
}
```



```
Problems 199   Output   Debug Console   Terminal   Ports   DevDb   Spell Checker 186   Azure

L\AppData\Roaming\Code\User\workspaceStorage\70dc8d60db77426e970e089c7cf34e2\redhat.java\jdt_ws\Daspr
Masukkan nilai n = 5
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
Total angka tercetak: 15
PS C:\Watkul\Daspro & Prakdaspro> 5
```

2. Tugas 2: Tabel Perkalian

Buat program untuk menampilkan tabel perkalian berukuran n × n menggunakan nested FOR atau WHILE. Nilai n minimal 3. Setiap posisi menampilkan hasil perkalian nomor baris dan nomor kolom. Atur spasi agar keluaran rapi, lalu tampilkan jumlah seluruh angka pada tabel.

Simpan program dengan nama file: **Tugas2NoAbsen.java**. Contoh untuk n = 4:

```
Masukkan nilai n =                    2
   1   2   3                         18
   2   4   6   3   6   9  12       Error!
Bookmark not defined.
```

```
   4   8  12  16
Jumlah seluruh angka: 100
```

Petunjuk: agar angka tersusun rata kanan, gunakan `System.out.printf("%4d", hasil);` yang mencetak bilangan dengan lebar 4 karakter, atau gu `"\t"` sebagai pemis

Jawaban:



```
import java.util.Scanner;

public class Tugas2_01 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Masukkan nilai n = ");
        int n = sc.nextInt();

        if (n < 3) {
            System.out.println("Nilai n harus minimal 3.");
        } else {
            int jumlahAngka = 0;

            for (int i = 1; i <= n; i++) {
                for (int j = 1; j <= n; j++) {
                    int hasil = i * j;

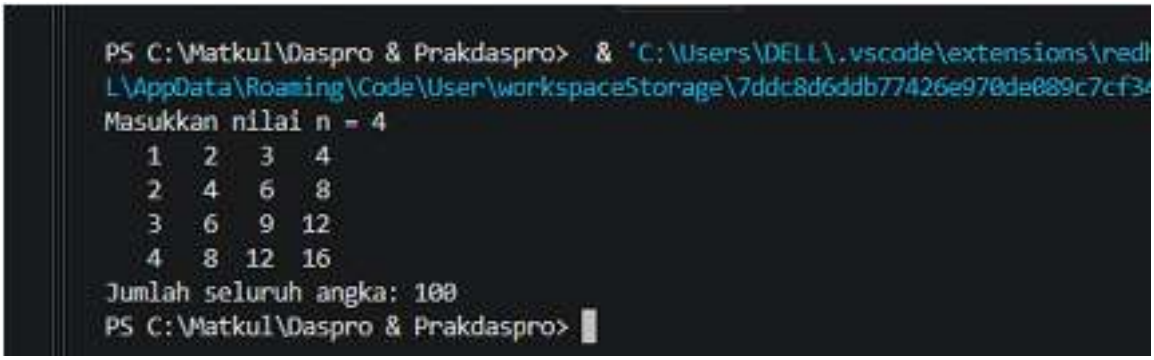
                    System.out.printf("%4d", hasil);

                    jumlahAngka += hasil;
                }

                System.out.println();
            }

            System.out.println("Jumlah seluruh angka: " + jumlahAngka);
        }

        sc.close();
    }
}
```



```
PS C:\Matkul\Daspro & Prakdaspro>  & 'C:\Users\DELL\.vscode\extensions\redh
L\AppData\Roaming\Code\User\workspaceStorage\7ddc8d6ddb77426e970de089c7cf34
Masukkan nilai n = 4
   1   2   3   4
   2   4   6   8
   3   6   9  12
   4   8  12  16
Jumlah seluruh angka: 100
PS C:\Matkul\Daspro & Prakdaspro>
```

3. Tugas 3: Rekap Penjualan Beberapa Toko

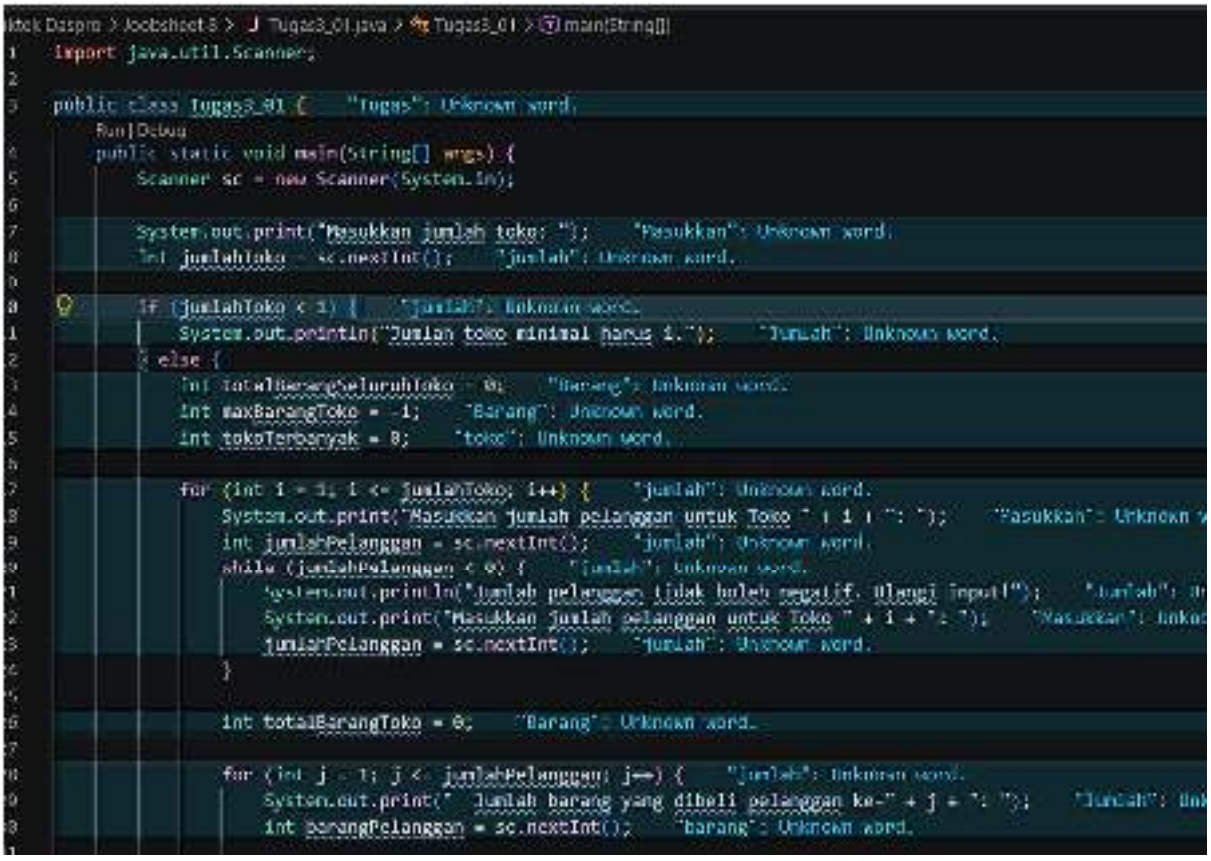
Sebuah pusat perbelanjaan memiliki beberapa toko. Setiap toko melayani sejumlah pelanggan pada hari yang sama, dan setiap pelanggan membeli beberapa barang. Program meminta jumlah toko, kemudian jumlah pelanggan pada masing-masing toko, dan jumlah barang yang dibeli setiap pelanggan. Hitung dan tampilkan total pelanggan serta total barang per toko, total barang seluruh toko, dan nomor toko dengan jumlah barang terbanyak. Jika terdapat nilai tertinggi yang sama, tampilkan toko dengan nomor paling kecil. Simpan program dengan nama file: **Tugas3NoAbsen.java.**

Ketentuan: jumlah toko minimal 1; jumlah pelanggan dan jumlah barang setiap pelanggan boleh 0, tetapi tidak boleh negatif. Gunakan perulangan bersarang; data tidak perlu disimpan satu per satu, cukup diakumulasikan ke variabel total.

Contoh: Toko 1 memiliki 2 pelanggan yang membeli 3 dan 2 barang; Toko 2 memiliki 3 pelanggan yang membeli 1, 4, dan 2 barang. Ringkasan yang diharapkan:

```
Toko 1 - pelanggan: 2, total barang: 5
Toko 2 - pelanggan: 3, total barang: 7
Total barang seluruh toko: 12
Toko dengan barang terbanyak: 2
```

Jawaban:





```
            for (int j = 1; j <= jumlahPelanggan; j++) {    "jumlah" tidak dikenal
                System.out.print("  Jumlah barang yang dibeli pelanggan ke-" + j + ": ");    "scanner" tidak dikenal
                int barangPelanggan = sc.nextInt();    "barang" tidak dikenal

                while (barangPelanggan < 0) {    "barang" tidak dikenal
                    System.out.println("  Jumlah barang tidak boleh negatif, ulangi input");    "jumlah" tidak dikenal
                    System.out.print("  Jumlah barang yang dibeli pelanggan ke-" + j + ": ");    "jumlah" tidak dikenal
                    barangPelanggan = sc.nextInt();    "barang" tidak dikenal
                }

                totalBarangToko += barangPelanggan;    "barang" tidak dikenal
            }

            System.out.println("Toko " + i + " - pelanggan: " + jumlahPelanggan + ", total barang: " + totalBarangToko);    "Toko

            totalBarangSeluruhToko += totalBarangToko;    "barang" tidak dikenal

            if (totalBarangToko > maxBarangToko) {    "barang" tidak dikenal
                maxBarangToko = totalBarangToko;    "barang" tidak dikenal
                tokoTerbanyak = i;    "toko" tidak dikenal
            }
        }

        System.out.println("=========================================");
        System.out.println("Total barang seluruh toko: " + totalBarangSeluruhToko);    "barang" tidak dikenal
        System.out.println("Toko dengan barang terbanyak: " + tokoTerbanyak);    "toko" tidak dikenal
    }

    sc.close();
  }
}
```

```
PS C:\Matkul\Daspro & Prakdaspro> ^C
PS C:\Matkul\Daspro & Prakdaspro> ^C
PS C:\Matkul\Daspro & Prakdaspro>
PS C:\Matkul\Daspro & Prakdaspro>  c:; cd 'c:\Matkul\Daspro & Prakdaspro'; 
InExceptionMessages' '-cp' 'C:\Users\DELL\AppData\Roaming\Code\User\workspa
Masukkan jumlah toko: 2
Masukkan jumlah pelanggan untuk Toko 1: 1
  Jumlah barang yang dibeli pelanggan ke-1: 2
Toko 1 - pelanggan: 1, total barang: 2
Masukkan jumlah pelanggan untuk Toko 2: 2
  Jumlah barang yang dibeli pelanggan ke-1: 1
  Jumlah barang yang dibeli pelanggan ke-2: 3
Toko 2 - pelanggan: 2, total barang: 4
=========================================
Total barang seluruh toko: 6
Toko dengan barang terbanyak: 2
PS C:\Matkul\Daspro & Prakdaspro>
```